Note: Write your code in the code cells, and your responses in markdown. Run the entire script and display the outputs of your code.

Due: **11:59PM Central Time on Friday, 09/26**. Upload both your code (.ipynb) and responses (html or pdf) to Canvas by then.

In [6]:
```
# Packages you might need: (pip install ... if you don't have them)
import pandas as pd # for data manipulation
import os  # for setting directory
print(os.getcwd())
# os.chdir() # input your personal directory where the dataset is saved
import statsmodels.formula.api as smf # for OLS regressions
import numpy as np  # to work with arrays (vectors/matrices)
```

```
/Users/alexdelatorre/Documents/School/UW-Madison/FALL 2025/ECON 695/Problem Sets/PS2
```

4 Oaxaca Decomposition

The goal of this problem set is to extend the analysis of public-private pay gaps in Lecture 3 to male workers. The file pay.csv contains 45,404 observations on male workers, age 30-50, with education of 12 or more years, in the March 2012 and 2013 CPS files. The variables are:

- govt=1 if government worker
- logwage=log hourly wage based on earnings, weeks worked, and average hours per week last year
- educ = years of education
- hs = 1 if high school grad (12 years of schooling)
- somecoll=1 if education is 13-14 years.
- college = 1 if BA (16 years of education)
- ma = 1 if masters degree (18 years of schooling)
- phd = 1 if phd or LLD or medical degree (20 years of schooling)
- age= age in years (range 30-50)

NOTE: note that “somecoll” combines 2 levels of education, 13 and 14 years (13 years are people with some college but no degree, 14 years are people with an AA degree from a community college).

Please create 2 new variables: educ13=1[educ=13] and educ14=1[educ=14] so you will have a total of 6 possible categories for education.

In [7]:
```
# Load dataset (using pandas "pd")
pay = pd.read_csv("pay.csv") # add your own directory if necessary

# create 2 new variables: educ13, educ14
```

```
pay['educ13'] = 0+(pay['educ']==13)
pay['educ14'] = 0+(pay['educ']==14)
print(pay.head(2))
# 6 educ categories: phd, ma, college, educ14, educ13, hs
# define a variable for 6 education levels:
pay['educ_level'] = 1*(pay['hs']==1) + 2*(pay['educ13']==1) + 3*(pay['educ14
print(pay['educ_level'].value_counts(dropna=False))
```

```
   age  phd  educ   logwage  hs  somecoll  college  ma  govt  educ13  educ14
0   39    0    14  2.890372   0         1        0   0     0       0       1
1   48    0    12  3.292984   1         0        0   0     0       0       0
educ_level
1    14310
4    11265
2     8199
3     5180
5     4437
6     2013
Name: count, dtype: int64
```

1. Means of All Variables in the private and public sectors

Construct a table like the 1st table (page 7) in Lecture 3. Show the means of all the variables for workers in the private (govt=0) and public (govt=1) sectors, and the differences in these means.

Hint: use pandas groupby https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.groupby.html, and reshape dataframes to look similar to the tables in lecture 3: (long to wide) https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.pivot.html

In [25]:
```
import numpy as np
import pandas as pd

# --- Part 1: Means of all variables by sector, nicely formatted ---

# 0) Which rows (variables) to show, and how to label them on the left
vars_to_show = ["logwage", "educ", "hs", "educ13", "educ14", "college", "ma"
row_labels = {
    "logwage": "Mean Log Wage",
    "educ":    "Mean Education",
    "hs":      "Fraction HS (12)",
    "educ13":  "Fraction 13 years",
    "educ14":  "Fraction 14 years",
    "college": "Fraction BA (16)",
    "ma":      "Fraction MA (18)",
    "phd":     "Fraction PhD (20)",
}

# 1) Group means for private (govt=0) and government (govt=1)
gm = (
    pay.groupby("govt", observed=True)[vars_to_show]
```

```
        .mean()
        .T  # variables as rows
        .rename(index=row_labels)
)

# 2) Build a wide, lecture-style table with grouped column headers
cols = pd.MultiIndex.from_tuples([
    ("Private Sector Workers",   "Mean"),
    ("Government Workers",       "Mean"),
    ("Public–Private Gap",       ""),   # gap = govt − private
])
table1_pretty = pd.DataFrame(index=gm.index, columns=cols)

table1_pretty[("Private Sector Workers", "Mean")] = gm[0]
table1_pretty[("Government Workers", "Mean")]   = gm[1]
table1_pretty[("Public–Private Gap", "")]       = gm[1] - gm[0]

# 3) Add the sample size row
n_by_sector = pay.groupby("govt", observed=True).size()
all_row = pd.Series({
    ("Private Sector Workers", "Mean"): n_by_sector.get(0, np.nan),
    ("Government Workers",   "Mean"):   n_by_sector.get(1, np.nan),
    ("Public–Private Gap",   ""):       np.nan,
}, name="Sample Size")

# 4) Final touches: order, labels, rounding, and display
table1_pretty.index.name = " "
table1_pretty = pd.concat([table1_pretty, all_row.to_frame().T], axis=0)

# Round everything except the sample-size row
num_rows = table1_pretty.index != "Sample Size"
table1_pretty.loc[num_rows] = table1_pretty.loc[num_rows].astype(float).rour
# Make sample sizes look like integers
table1_pretty.loc["Sample Size"] = table1_pretty.loc["Sample Size"].astype("

display(table1_pretty)
```

	Private Sector Workers	Government Workers	Public–Private Gap
	Mean	Mean	
Mean Log Wage	3.007	3.219	0.212
Mean Education	14.212	15.073	0.861
Fraction HS (12)	0.337	0.192	-0.146
Fraction 13 years	0.180	0.182	0.002
Fraction 14 years	0.113	0.121	0.008
Fraction BA (16)	0.246	0.258	0.012
Fraction MA (18)	0.083	0.180	0.097
Fraction PhD (20)	0.040	0.067	0.027
Sample Size	38553.000	6851.000	NaN

2. Education and Wage Gaps

Construct a table like the 2nd table (page 8) in Lecture 3. Show the fractions of private and government workers at each education level, and the mean wages of the two groups in each education level, and the public-private wage gap at each level.

In [22]:
```
# --- Table 2: Education-specific composition and wage gaps (single wide tab

import numpy as np
import pandas as pd
from IPython.display import display

# (Optional) keep wide tables on one line in VS Code / Jupyter
pd.set_option('display.expand_frame_repr', False)

# 1) A single education code: 12, 13, 14, 16, 18, 20
edu_code = np.select(
    [
        pay['hs'].eq(1),        # 12
        pay['educ13'].eq(1),    # 13
        pay['educ14'].eq(1),    # 14
        pay['college'].eq(1),   # 16
        pay['ma'].eq(1),        # 18
        pay['phd'].eq(1),       # 20
    ],
    [12, 13, 14, 16, 18, 20],
    default=np.nan
)
cats = [12, 13, 14, 16, 18, 20]
pay['edu_code'] = pd.Categorical(edu_code, categories=cats, ordered=True)

# 2) Keep only rows with a valid edu_code
df = pay.dropna(subset=['edu_code']).copy()
```

```python
# 3) Group by (education, sector) and compute counts and mean log wages
g = df.groupby(['edu_code', 'govt'], observed=True)
by_cell = g.agg(
    n=('logwage', 'size'),
    mean_logw=('logwage', 'mean')
).reset_index()

# 4) Fractions are "within sector": for each sector, the fraction of that se
by_cell['sector_total'] = by_cell.groupby('govt')['n'].transform('sum')
by_cell['fraction_in_group'] = by_cell['n'] / by_cell['sector_total']

# 5) Pivot to wide
wide_w = by_cell.pivot(index='edu_code', columns='govt', values='mean_logw')
wide_f = by_cell.pivot(index='edu_code', columns='govt', values='fraction_ir

# 6) Assemble table with clear column order and a MultiIndex header like the
cols = pd.MultiIndex.from_tuples([
    ('Private Sector Workers',   'Mean Log Wage'),
    ('Private Sector Workers',   'Fraction in Group'),
    ('Government Workers',       'Mean Log Wage'),
    ('Government Workers',       'Fraction in Group'),
    ('Govt Wage Premium',        '')  # (govt - private)
])

table2 = pd.DataFrame({
    ('Private Sector Workers', 'Mean Log Wage'):   wide_w.get(0),
    ('Private Sector Workers', 'Fraction in Group'): wide_f.get(0),
    ('Government Workers',     'Mean Log Wage'):   wide_w.get(1),
    ('Government Workers',     'Fraction in Group'): wide_f.get(1),
})

# Wage premium: govt minus private, by education
table2[('Govt Wage Premium', '')] = (
    table2[('Government Workers', 'Mean Log Wage')] -
    table2[('Private Sector Workers', 'Mean Log Wage')]
)

# 7) Add the “All” row
overall = df.groupby('govt').agg(
    n=('logwage', 'size'),
    mean_logw=('logwage', 'mean')
)
all_row = pd.Series({
    ('Private Sector Workers', 'Mean Log Wage'):   overall.loc[0, 'mean_logw
    ('Private Sector Workers', 'Fraction in Group'): 1.0,   # within-sector
    ('Government Workers',     'Mean Log Wage'):   overall.loc[1, 'mean_logw
    ('Government Workers',     'Fraction in Group'): 1.0,   # within-sector
    ('Govt Wage Premium',      ''): (overall.loc[1, 'mean_logw'] - overall.l
       if (0 in overall.index and 1 in overall.index) else np.nan
}, name='All')

# 8) Final touches: order, labels, rounding, show
table2.columns = cols
table2.index = table2.index.astype('object')  # so we can append 'All'
table2 = pd.concat([table2, all_row.to_frame().T], axis=0)
```

```
table2.index.name = 'Education'
table2 = table2.round(3)

display(table2)
```

	Private Sector Workers		Government Workers		Govt Wage Premium
	Mean Log Wage	Fraction in Group	Mean Log Wage	Fraction in Group	
Education					
12	2.703	0.337	2.968	0.192	0.265
13	2.872	0.180	3.122	0.182	0.250
14	2.935	0.113	3.157	0.121	0.222
16	3.257	0.246	3.274	0.258	0.017
18	3.554	0.083	3.425	0.180	-0.129
20	3.692	0.040	3.543	0.067	-0.149
All	3.007	1.000	3.219	1.000	0.212

3. Distribution across Education Categories, by Group

Using the notation of Lecture 3, let private sector workers be called "group a" (who are the reference group), let government workers be called "group b", and define the 6 category variables D_{gi} for each level of education g=1...6. Let $x_i' = (D_{1i}, D_{2i}, \ldots D_{6i})$. Find $\overline{x}^a$ and $\overline{x}^b$.

In [26]:
```
# --- Define groups (a = private, b = govt) and education dummies D_gi ---

import numpy as np
import pandas as pd

# Group indicator: a = private (govt=0), b = govt (govt=1)
pay['group'] = np.where(pay['govt'] == 1, 'b', 'a')

# Create the six education category dummies
pay['D_12'] = (pay['hs'] == 1).astype(int)          # High school grad (12 yr
pay['D_13'] = (pay['educ'] == 13).astype(int)      # Some college, no degree
pay['D_14'] = (pay['educ'] == 14).astype(int)      # AA degree (14 yrs)
pay['D_16'] = (pay['college'] == 1).astype(int)    # BA (16 yrs)
pay['D_18'] = (pay['ma'] == 1).astype(int)         # MA (18 yrs)
pay['D_20'] = (pay['phd'] == 1).astype(int)        # PhD/MD/LLD (20 yrs)

# Collect dummy names for later use
D_cols = ['D_12','D_13','D_14','D_16','D_18','D_20']
```

```
# Quick check: each observation should have exactly one dummy = 1
pay[D_cols].sum(axis=1).value_counts()
```

Out[26]:
```
1    45404
Name: count, dtype: int64
```

(a) OLS: Log wage on education categories, by group

Fit an OLS regression model for wages on x_i for each group, and estimate the coefficients $\hat{\beta}^a$ and $\hat{\beta}^b$. Verify that these coefficients are the mean log wages of groups a and b, respectively, for each level of education.

Hint: see reg_dummy.py under Files/Lectures/lecture2 for example code. I used: smf.ols("y ~ 0 + C(x)", data=temp).fit(cov_type='HC1'), where 0 suppresses constant, x is a categorical variable (so the regression coefficients would be means of each category).

In [28]:
```
# --- Part 3(a): OLS of log wages on education dummies, by group ---

import numpy as np
import pandas as pd
import statsmodels.formula.api as smf

# 0) Make sure we have the split-at-13/14 and an ordered edu_code
if 'educ13' not in pay.columns:
    pay['educ13'] = (pay['educ'] == 13).astype(int)
if 'educ14' not in pay.columns:
    pay['educ14'] = (pay['educ'] == 14).astype(int)

if 'edu_code' not in pay.columns:
    edu_code = np.select(
        [
            pay['hs'].eq(1),       # 12
            pay['educ13'].eq(1),   # 13
            pay['educ14'].eq(1),   # 14
            pay['college'].eq(1),  # 16
            pay['ma'].eq(1),       # 18
            pay['phd'].eq(1),      # 20
        ],
        [12, 13, 14, 16, 18, 20],
        default=np.nan
    )
    pay['edu_code'] = pd.Categorical(edu_code, categories=[12,13,14,16,18,20

# 1) Ensure the six education dummies D_g exist (D_12,...,D_20)
D_cols = [f'D_{g}' for g in [12,13,14,16,18,20]]
for g in [12,13,14,16,18,20]:
    col = f'D_{g}'
    if col not in pay.columns:
        pay[col] = (pay['edu_code'] == g).astype(int)

# 2) Split data: a = private (govt=0), b = government (govt=1)
a = pay.loc[pay['govt'] == 0].copy()
```

```
b = pay.loc[pay['govt'] == 1].copy()

# 3) OLS with no intercept so each coefficient = cell mean
formula = "logwage ~ 0 + D_12 + D_13 + D_14 + D_16 + D_18 + D_20"
fit_a = smf.ols(formula, data=a).fit(cov_type='HC1')
fit_b = smf.ols(formula, data=b).fit(cov_type='HC1')

# 4) Collect coefficients and relabel index to numeric education codes
beta_a = fit_a.params.rename(index=lambda s: int(s.split('_')[1])).sort_inde
beta_b = fit_b.params.rename(index=lambda s: int(s.split('_')[1])).sort_inde

# 5) Direct group-by means for verification
mean_a = a.groupby('edu_code', observed=True)['logwage'].mean().reindex(beta
mean_b = b.groupby('edu_code', observed=True)['logwage'].mean().reindex(beta

# 6) Sanity checks: OLS coefs == means (up to numerical noise)
assert np.allclose(beta_a.values, mean_a.values), "Private (a) coefficients
assert np.allclose(beta_b.values, mean_b.values), "Government (b) coefficier

# 7) Display a tidy comparison table (with plain-English headers)
out_a_b = pd.DataFrame({
    'OLS coef (a)': beta_a.round(3),
    'Mean log wage (a)': mean_a.round(3),
    'OLS coef (b)': beta_b.round(3),
    'Mean log wage (b)': mean_b.round(3),
})
out_a_b.index.name = 'Education'
display(out_a_b)

# 8) (Stored for later parts) vectors of coefficients and x̄^a, x̄^b
beta_a_vec = beta_a.reindex([12,13,14,16,18,20]).to_numpy()
beta_b_vec = beta_b.reindex([12,13,14,16,18,20]).to_numpy()
xbar_a = a[D_cols].mean().rename(index=lambda s: int(s.split('_')[1])).reinc
xbar_b = b[D_cols].mean().rename(index=lambda s: int(s.split('_')[1])).reinc
```

Education	OLS coef (a)	Mean log wage (a)	OLS coef (b)	Mean log wage (b)
12	2.703	2.703	2.968	2.968
13	2.872	2.872	3.122	3.122
14	2.935	2.935	3.157	3.157
16	3.257	3.257	3.274	3.274
18	3.554	3.554	3.425	3.425
20	3.692	3.692	3.543	3.543

(b) Mean log wage as a weighted average

Using your estimates of $\bar{x}^a$, $\bar{x}^b$, $\hat{\beta}^a$ and $\hat{\beta}^b$, construct $\bar{y}^a = \sum_g \bar{p}_g^a \bar{y}_g^a = (\bar{x}^a)'\hat{\beta}^a$ and $\bar{y}^b = \sum_g \bar{p}_g^b \bar{y}_g^b = (\bar{x}^b)'\hat{\beta}^b$. Verify that these match the means of wages for groups a and

b that you construct directly.

In [29]:

```
# --- Q3(b): Mean log wage as a weighted average (law of iterated expectatic

import numpy as np
import pandas as pd
import statsmodels.formula.api as smf

# 1) Split groups a (private) and b (government)
a = pay.loc[pay['govt'].eq(0)].copy()
b = pay.loc[pay['govt'].eq(1)].copy()

# 2) Education-category dummies (D_12,...,D_20).
#    (If you already created these above, this just re-asserts them.)
pay['D_12'] = (pay['hs']      == 1).astype(int)
pay['D_13'] = (pay['educ']    == 13).astype(int)
pay['D_14'] = (pay['educ']    == 14).astype(int)
pay['D_16'] = (pay['college'] == 1).astype(int)
pay['D_18'] = (pay['ma']      == 1).astype(int)
pay['D_20'] = (pay['phd']     == 1).astype(int)

a[['D_12','D_13','D_14','D_16','D_18','D_20']] = pay.loc[a.index, ['D_12','D
b[['D_12','D_13','D_14','D_16','D_18','D_20']] = pay.loc[b.index, ['D_12','D

# Nice ordered list of the dummy names
D_cols = ['D_12','D_13','D_14','D_16','D_18','D_20']

# 3) Fit the dummy-only OLS in each group (0 eliminates the intercept so coe
mod_a = smf.ols("logwage ~ 0 + D_12 + D_13 + D_14 + D_16 + D_18 + D_20", dat
mod_b = smf.ols("logwage ~ 0 + D_12 + D_13 + D_14 + D_16 + D_18 + D_20", dat

beta_a = mod_a.params[D_cols]     # \hat{β}^a_g = mean logwage in group a, c
beta_b = mod_b.params[D_cols]     # \hat{β}^b_g = mean logwage in group b, c

# 4) Build \bar{x}^a and \bar{x}^b (fractions in each education category wit
xbar_a = a[D_cols].mean()         # p_g^a
xbar_b = b[D_cols].mean()         # p_g^b

# 5) Weighted-average means:  \bar{y}^a = (\bar{x}^a)' \hat{β}^a  and  \bar{
ybar_a_weighted = float(np.dot(xbar_a.values, beta_a.values))
ybar_b_weighted = float(np.dot(xbar_b.values, beta_b.values))

# 6) Direct sample means (for verification)
ybar_a_direct = a['logwage'].mean()
ybar_b_direct = b['logwage'].mean()

# 7) Tidy check table
check = pd.DataFrame({
    "Weighted avg (x̄'β̂)": [ybar_a_weighted, ybar_b_weighted],
    "Direct mean logwage":  [ybar_a_direct,   ybar_b_direct],
    "Difference":           [ybar_a_weighted - ybar_a_direct,
                             ybar_b_weighted - ybar_b_direct]
}, index=pd.Index(['Group a (private)','Group b (government)'], name='Group'

display(check)
```

```
# 8) (Optional) assert closeness
assert np.isclose(ybar_a_weighted, ybar_a_direct), "Group a: weighted avg !=
assert np.isclose(ybar_b_weighted, ybar_b_direct), "Group b: weighted avg !=
```

Group	Weighted avg (x̄'β̂)	Direct mean logwage	Difference
Group a (private)	3.007	3.007	-0.0
Group b (government)	3.219	3.219	0.0

(c) Counterfactual 1

Using your estimates of $\bar{x}^a$, and $\hat{\beta}^b$ construct

$$\bar{y}^b_{counterf} = \sum_g \bar{p}^a_g \bar{y}^b_g = (\bar{x}^a)'\hat{\beta}^b$$

Find the adjusted wage gap: $\bar{y}^b_{counterf} - \bar{y}^a$. How does this compare to the actual gap $\bar{y}^b - \bar{y}^a$?

In [30]:
```
# --- 3(c) Counterfactual using a's composition and b's returns ---

import numpy as np
import pandas as pd
import statsmodels.formula.api as smf

# Dummies (should already exist from part 3 setup)
D_cols = ['D_12','D_13','D_14','D_16','D_18','D_20']
order  = [f'D_{g}' for g in [12,13,14,16,18,20]]

# Split data
a = pay.loc[pay['govt'].eq(0)].copy()  # private (group a)
b = pay.loc[pay['govt'].eq(1)].copy()  # government (group b)

# 1) Returns for group b: \hat{beta}^b from a no-constant dummy regression
formula_b = "logwage ~ 0 + " + " + ".join(D_cols)
fit_b = smf.ols(formula_b, data=b).fit(cov_type='HC1')
beta_b = fit_b.params.reindex(order)

# 2) Composition for group a: xbar^a (mean of education dummies)
xbar_a = a[D_cols].mean().reindex(order)

# 3) Counterfactual mean for group b under a's composition
ybar_b_counterf = float(np.dot(xbar_a.values, beta_b.values))

# 4) Actual means and gaps
ybar_a = a['logwage'].mean()
ybar_b = b['logwage'].mean()

gap_actual   = ybar_b - ybar_a
```

```
gap_adjusted = ybar_b_counterf - ybar_a

# (Optional) Oaxaca-style split when using b-coefficients as reference
# composition piece: (xbar^b - xbar^a)' * beta^b
xbar_b = b[D_cols].mean().reindex(order)
composition = float(np.dot((xbar_b - xbar_a).values, beta_b.values))
# returns piece: (beta^b - beta^a)' * xbar^a   (beta^a from part 3(a); recom
fit_a = smf.ols("logwage ~ 0 + " + " + ".join(D_cols), data=a).fit(cov_type=
beta_a = fit_a.params.reindex(order)
returns = float(np.dot((beta_b - beta_a).values, xbar_a.values))

# 5) Display tidy summary
out = pd.DataFrame(
    {
        "Mean log wage": [ybar_a, ybar_b, ybar_b_counterf],
        "_": ["", "", ""],
        "Gap vs group a": [0.0, gap_actual, gap_adjusted],
    },
    index=["Group a (private)", "Group b (govt) — actual", "Group b — counte
)

pieces = pd.DataFrame(
    {
        "Composition piece (using β^b)": [composition],
        "Returns piece (using x̄^a)": [returns],
        "Composition + Returns": [composition + returns],
        "Actual gap ȳ^b - ȳ^a": [gap_actual],
    }
)

display(out.round(3))
display(pieces.round(3))
```

	Mean log wage	—	Gap vs group a
Group a (private)	3.007		0.000
Group b (govt) – actual	3.219		0.212
Group b – counterfactual	3.154		0.147

	Composition piece (using β^b)	Returns piece (using x̄^a)	Composition + Returns	Actual gap ȳ^b – ȳ^a
0	0.065	0.147	0.212	0.212

Interpretation of Part (c)

The actual observed wage gap between government and private sector workers is

$$\bar{y}^b - \bar{y}^a = 3.219 - 3.007 = 0.212.$$

Using the Oaxaca decomposition, we construct the counterfactual mean log wage for government workers under the private-sector education distribution:

$$\bar{y}^{b}_{\text{counterf}} \;=\; 3.154.$$

This yields an adjusted wage gap of

$$\bar{y}^{b}_{\text{counterf}} - \bar{y}^{a} \;=\; 3.154 - 3.007 \;=\; 0.147.$$

Decomposing the total gap of (0.212), we find:

- **Composition effect (education mix):** (0.065)
- **Returns effect (differences in pay for the same education):** (0.147)

Thus, approximately (31%) of the wage gap is attributable to differences in the education distribution across sectors, while the remaining (69%) reflects differences in returns to education between government and private-sector employment.

(d) Counterfactual 2

Construct the weight $w_g = N^a_g / N^b_g$ for education categories $g = 1...6$. Use this to construct $\bar{y}^{b}_{counterf2} = \frac{\sum_{i\in b} w_g y_i}{\sum_{i\in b} w_g}$ and verify that:

$$\bar{y}^{b}_{counterf} = \sum_g \bar{p}^{a}_{g}\bar{y}^{b}_{g} = (\bar{x}^{a})'\hat{\beta}^{b} = \frac{\sum_{i\in b} w_g y_i}{\sum_{i\in b} w_g} = \bar{y}^{b}_{counterf2}$$

.

In [32]:
```
import numpy as np
import pandas as pd

# --- (d) Counterfactual 2: cell weights and weighted mean ---

# Counts by education cell in each group
Na_g = a[D_cols].sum(axis=0)   # number of a (private) in each edu cell
Nb_g = b[D_cols].sum(axis=0)   # number of b (govt)   in each edu cell

# Weights w_g = Na_g / Nb_g (defined cell-by-cell)
w_g = (Na_g / Nb_g).rename('w_g')

# Attach weights to each person in group b according to their edu cell
# For each person i in b, w_i = sum_g w_g * D_gi  (since exactly one D_gi=1)
w_i = pd.Series(0.0, index=b.index)
for col in D_cols:
    w_i = w_i + w_g[col] * b[col]

# Weighted mean of log wages for group b under weights w_g:
numer = (w_i * b['logwage']).sum()
denom = w_i.sum()
ybar_b_counterf2 = numer / denom

# Compare to the counterfactual from (c): (xbar^a)' beta^b
# (Assumes you already computed ybar_b_counterf)
```

```
diff_cf = ybar_b_counterf2 - ybar_b_counterf

# Adjusted gap using the cell-weighted counterfactual
gap_adj_d = ybar_b_counterf2 - ybar_a_direct

# Tidy display
check_d = pd.DataFrame({
    "ybar_b (counterf2)": [ybar_b_counterf2],
    "ybar_b (xbar^a beta^b)": [ybar_b_counterf],
    "Difference": [diff_cf],
    "Adjusted gap (counterf2 - ybar_a)": [gap_adj_d]
})
display(check_d.round(3))
```

	ybar_b (counterf2)	ybar_b (xbar^a beta^b)	Difference	Adjusted gap (counterf2 - ybar_a)
0	3.154	3.154	-0.0	0.147

Interpretation of Part (d)

In Part (d), we constructed the counterfactual mean log wage for government workers using **cell weights** defined as:

$$w_g = \frac{N_g^a}{N_g^b}, \quad g = 1, \ldots, 6$$

where (N^a_g) and (N^b_g) are the number of workers in education category (g) for the private and government sectors, respectively.

The weighted counterfactual mean is:

$$\bar{y}^b_{\text{counterf2}} = \frac{\sum_{i \in b} w_g y_i}{\sum_{i \in b} w_g} = 3.154$$

We then verified that this equals the alternative construction from Part (c):

$$\bar{y}^b_{\text{counterf2}} = (\bar{x}^a)' \hat{\beta}^b = 3.154$$

with a difference of essentially **0.0**, confirming that both methods are consistent.

Finally, the adjusted wage gap using this counterfactual is:

$$\bar{y}^b_{\text{counterf2}} - \bar{y}^a = 3.154 - 3.007 = 0.147$$

which matches the *returns* component identified earlier in the Oaxaca decomposition. Thus, Part (d) confirms that the decomposition result is robust to the alternative construction of the counterfactual mean.